

Comprehensive Engineering & Knowledge Transfer Report

TataStrive Analytics v3.0.0

Internship Handover Document

June 19, 2026

Contents

1	Executive Summary & Business Impact	4
2	Intern Onboarding: Glossary & Core Concepts	4
3	System Architecture & Application Design	5
3.1	Frontend & Concurrency Model	5
3.2	Idempotency & State Management	5
4	The Machine Learning & Tracking Pipeline	5
4.1	Tracking Pivot: From ByteTrack to BoTSORT	5
4.2	Identity Fragmentation & Post-Stitching Logic	5
5	Enterprise Data Warehouse Integration (BigQuery)	7
5.1	BigQuery Schema & Synchronization	7
5.2	Data Extraction Query Architecture	7
6	Optimization: Time & Overhead Reduction Methods	7
7	The Automated Verification & Audit Pipeline	8
7.1	Phase 1: Overcoming Local I/O & Bandwidth Bottlenecks	8
7.2	Phase 2: Fixing S3 Object Structure Bugs	8
7.3	Phase 3: The NVR Time Drift & 6-Second Clip Paradigm	9
8	Operational Playbooks & Deployment Strategy	10
8.1	Executable Freezing & Silent Updates	10
8.2	Hardware Optimization Playbook	10
9	Current Limitations & Diagnostic Playbook	10
9.1	Error Type A: The “Ghost” (Detection Failure)	10
9.2	Error Type B: The “Visitor Mismatch” (Recognition Failure)	11
10	Milestones, Successes, & Key Learnings	12
10.1	Project Milestones	12
10.2	Failures & Strategic Pivots	12
10.3	Key Engineering Learnings	12
11	Future Scope & Roadmap	13
11.1	1. Native OCR Timestamp Synchronization	13
11.2	2. Automated Re-Enrollment (The “Frequent Visitor” Pipeline)	13
11.3	3. Edge-to-Cloud Inference Migration	13
A	Access Management & Quick Start	14
B	Operator Runbook: New Center Deployment	15
B.1	Phase 1: Environment & Hardware Verification	15
B.2	Phase 2: Installation & Configuration	15
B.3	Phase 3: Pipeline Activation	15

1 Executive Summary & Business Impact

The Business Problem: TataStrive requires a highly scalable, automated method to track student attendance and classroom engagement without manual intervention. The previous manual systems were prone to human error and difficult to audit at scale.

The Solution: My team deployed an AI-powered desktop application to edge PCs in the training centers. This application processes CCTV footage locally, identifies students, and uploads lightweight, structured attendance data directly to Google BigQuery for leadership dashboards.

Internship Impact & ROI: A significant challenge my team faced was verifying the AI's accuracy. Manually downloading gigabytes of annotated video from Google Drive to audit a single student took hours. We engineered a cloud-native auditing pipeline (via Google Colab) that dynamically fetches AWS S3 profile pictures and generates 6-second verification clips. **This reduced the time required to audit a center's daily attendance from hours to minutes**, providing a scalable framework to continuously tune the AI's accuracy and directly impacting the reliability of the organization's data.

2 Intern Onboarding: Glossary & Core Concepts

If you are new to Computer Vision (CV) or this project, here is how our pipeline works in plain English:

- **YOLOv8 (You Only Look Once):** The AI model that acts as the “eyes”. It scans a video frame and draws a bounding box around anything that looks like a person or a face.
- **Tracking & IoU (Intersection over Union):** Once YOLO finds a person, we need to track them across frames. IoU is simply math that checks how much a bounding box in Frame 1 overlaps with a bounding box in Frame 2. If they overlap significantly, the AI assumes it is the same person.
- **BoTSORT & Re-ID (Re-Identification):** IoU fails if a student walks behind a pillar. BoTSORT fixes this by using a Re-ID model, which remembers the “appearance” (like shirt color) of the student so it can re-identify them when they step out from behind the pillar.
- **InsightFace & Embeddings:** When the AI crops a face, InsightFace turns that face into an “Embedding”—a list of 512 numbers that acts as a unique mathematical fingerprint.
- **Cosine Similarity:** To check if the CCTV face matches the AWS S3 database photo, we calculate the “Cosine Similarity” between their two embeddings. A score of 1.0 is a perfect match. We currently consider anything above 0.40 a confirmed student.

3 System Architecture & Application Design

TataStrive Analytics v3.0.0 is a robust, Windows-focused desktop application designed to process raw CCTV footage.

3.1 Frontend & Concurrency Model

The user interface is built using `PyQt6` to provide operators with a familiar desktop environment. To prevent the UI from freezing during intensive machine learning inference, the application heavily utilizes the `QThread` concurrency model.

- **ClassroomWorker** (`app/workers/classroom_worker.py`): Handles the engagement pipeline, emitting signals for progress, logs, and video preview frames.
- **CrossDayWorker** (`app/workers/crossday_worker.py`): Wraps the attendance tracking logic. It evaluates tracking thresholds (e.g., `t_strict_merge`, `t_new_id`), updates the master `.pkl/.db` database, and manages annotated video generation.

3.2 Idempotency & State Management

A core requirement for center operators is zero-touch monitoring. The system achieves this via a folder-listening mechanism. The `ConfigManager` (`app/config.py`) maintains a persistent JSON state file located at `~/.tatastrive/config.json`. This file stores `crossday_completed_videos`, ensuring that if the application crashes or restarts, it never re-processes the same `.mp4` file.

4 The Machine Learning & Tracking Pipeline

4.1 Tracking Pivot: From ByteTrack to BoTSORT

Early in the project lifecycle, **ByteTrack** was utilized for multi-object tracking. However, it exhibited severe limitations:

- **The Failure Mode:** ByteTrack relies heavily on IoU. In a dense classroom environment, when students crossed paths or walked behind pillars (occlusion), the bounding boxes disappeared. Upon reappearing, ByteTrack assigned a completely new Identity (ID).
- **The BoTSORT Solution:** We migrated to BoTSORT (via the `boxmot` library). By extracting visual features of the student's clothing alongside Kalman filter motion prediction, BoTSORT successfully maintained the `G_ID` even after multi-second occlusions.

4.2 Identity Fragmentation & Post-Stitching Logic

Even with BoTSORT, extreme occlusions (e.g., a student leaving the room and returning 10 minutes later) caused fragmented tracking IDs (e.g., Student A is tracked as `G_009`, leaves, and returns as `G_143`).

To resolve this, my team implemented the **Post-Stitching Algorithm** (`classroom_analysis/stitching.py`). This logic groups tracks by evaluating two configurable parameters from the settings dialog:

1. **Max Time Gap:** The maximum allowable temporal distance between the end of one track and the start of another.
2. **Max Pixel Distance:** The spatial bounding box coordinates. If G_009 vanished at coordinate (x, y) , and G_143 appeared at coordinate (x, y) within the **Max Time Gap**, the system dynamically merges them into a single continuous attendance session.

5 Enterprise Data Warehouse Integration (BigQuery)

Processing edge-video locally is only half the architecture. The structured JSON outputs must be synchronized to a central data warehouse to allow organizational leaders to run cross-center analytics.

5.1 BigQuery Schema & Synchronization

The synchronization logic resides in `app/bigquery_sync.py`. The application flattens nested local JSON reports into relational structures mapped to three primary BigQuery tables:

- `attendance_reports`: Contains granular data for every tracked individual.
- `engagement_reports`: Summarizes classroom dynamics.
- `sync_log`: An audit trail recording any files that failed to process.

Every single row uploaded to BigQuery is injected with the `center_id` (configured during the app's first launch) to allow multi-tenant querying.

5.2 Data Extraction Query Architecture

During our auditing phase, we needed to extract a "Ground Truth" list of active students. Initially, standard SQL date math (`start_date <= test_date`) failed silently because the upstream CRM stored dates as raw strings (e.g., "15-01-2026").

We engineered a bulletproof extraction query utilizing BigQuery's `PARSE_DATE` engine to cast the strings into chronological data types dynamically:

```
1 SELECT DISTINCT
2     engagement_id,
3     batch_name
4 FROM
5     'tatastrive_project.analytics.students_intraining'
6 WHERE
7     PARSE_DATE('%d-%m-%Y', start_date) <= '2026-02-05'
8     AND PARSE_DATE('%d-%m-%Y', end_date) >= '2026-02-04'
9     AND engagement_id IS NOT NULL;
```

Listing 1: Ground Truth Extraction Query

6 Optimization: Time & Overhead Reduction Methods

Processing gigabytes of video locally required aggressive resource management.

1. **Frame Skipping:** Processing every 30 FPS was redundant. We implemented a `Default Frame Skip` parameter. Analyzing 1-3 frames per second yielded nearly identical embedding accuracy while slashing inference time by 90%.

2. **Disabling Live Previews:** The `PyQt6 Video Preview` consumed heavy CPU/GPU rendering cycles. Disabling this during batch processing shifted all compute power to the YOLO/InsightFace tensors.
3. **I/O Overhead Reduction (Colab Migration):** Downloading massive NVR .mp4 folders to local machines for auditing was mathematically unfeasible. We shifted our verification scripts to Google Colab, executing code directly against mounted Google Drive folders.
4. **Storage I/O Batching:** Writing hundreds of face crops directly to a mounted Google Drive caused severe latency and `[Errno 2]` crashes. We bypassed this by caching outputs to Colab's local high-speed disk (`/content/`) and executing a single `shutil.make_archive` zip operation at the very end.

7 The Automated Verification & Audit Pipeline

The most significant engineering challenge during this internship was auditing the accuracy of the AI. Given thousands of hours of video and tens of thousands of face matches, manual verification was impossible. We developed a highly sophisticated, cloud-native auditing pipeline utilizing Google Colab.

7.1 Phase 1: Overcoming Local I/O & Bandwidth Bottlenecks

Our initial approach involved downloading the annotated output videos from Google Drive to local laptops and using OpenCV to extract single frames. This failed due to massive bandwidth requirements and slow local I/O.

The Colab Pivot: We migrated the audit environment to Google Colab. By mounting the organization's Google Drive directly into the Colab runtime (`/content/drive`), we achieved server-side processing speeds, completely bypassing local network downloads.

7.2 Phase 2: Fixing S3 Object Structure Bugs

To verify a match, the script needed to fetch the student's master profile picture from AWS S3 using `boto3`.

- **The 0-Byte Folder Bug:** S3 treats folders as 0-byte objects. Our script initially crashed because `Contents[0]` grabbed the folder itself instead of the image.
- **The ID Card Bug:** Many student S3 folders contained multiple uploads (Aadhar cards, old thumbnails).
- **The Solution:** We implemented a strict filtering algorithm to iterate over the bucket contents, explicitly requiring valid image extensions and the `_StudentPicture` substring.

```
1 def fetch_s3_image(engagement_id, download_path):
2     prefix = f"{engagement_id}/"
3     response = s3.list_objects_v2(Bucket=S3_BUCKET, Prefix=prefix)
4
5     valid_extensions = ('.jpg', '.jpeg', '.png')
```



```

6     s3_key = None
7
8     for obj in response.get('Contents', []):
9         file_key = obj['Key'].lower()
10        if file_key.endswith(valid_extensions) and "_studentpicture" in
11        file_key:
12            s3_key = obj['Key']
13            break
14
15        if not s3_key:
16            raise ValueError(f"No valid master image found for {
17        engagement_id}")
18
19    s3.download_file(S3_BUCKET, s3_key, download_path)

```

Listing 2: Hardened S3 Image Fetcher

7.3 Phase 3: The NVR Time Drift & 6-Second Clip Paradigm

Our initial auditing script took a static screenshot of the CCTV video based on the `entry_time` in the BigQuery CSV.

The Critical Failure: Network Video Recorders (NVRs) exhibit *Metadata Lag*. The timestamp written into the `.mp4` filename often lags behind the camera’s physical clock by 3 to 5 seconds. Furthermore, security cameras use Variable Frame Rates (VFR) to save space. Mathematical frame calculation ($seconds \times FPS$) drifted wildly, resulting in screenshots of empty hallways or the backs of the wrong people.

The 6-Second Clip Solution: We abandoned single-frame extraction. We engineered a Colab pipeline that utilizes `cv2.VideoWriter` to extract a 6-second window (3 seconds prior to the timestamp, 3 seconds after). This temporal buffer guaranteed that the human auditor would see the student walking into the frame, regardless of NVR drift.

Furthermore, to prevent Google Drive rate-limit crashes ([Errno 2]), the script renders the videos to Colab’s high-speed local disk and zips them into a single archive before uploading back to Drive.

8 Operational Playbooks & Deployment Strategy

Deploying a heavy computer vision application to remote training centers with varying hardware capabilities required a robust deployment and update strategy.

8.1 Executable Freezing & Silent Updates

- **PyInstaller & Inno Setup:** The application is bundled into a standalone Windows executable using PyInstaller (`build_exe.py`), packaged within an Inno Setup installer. This removes the need for center administrators to manage Python environments or pip dependencies.
- **The Silent Updater:** To push model weight updates or threshold tweaks without requiring a manual reinstall, we built a silent updater (`app/updater.py`). The application periodically polls the GitHub Releases API. If a new version is detected, it downloads a delta ZIP, extracts it over the installation directory, clears the `--pycache--`, and automatically restarts the process.

8.2 Hardware Optimization Playbook

Many remote centers run on low-spec PCs. We built configurable fail-safes into the UI to prevent CUDA Out-Of-Memory (OOM) crashes:

1. **CPU Fallback:** If PyTorch GPU initialization fails (due to missing drivers), the application seamlessly falls back to CPU inference.
2. **Aggressive Frame Skipping:** Operators on weak PCs are trained to increase the `Default Frame Skip` setting, sacrificing temporal granularity to maintain system stability.
3. **Disabling Previews:** Rendering bounding boxes to the PyQt6 UI consumes significant resources. Operators disable the `Enable Preview` checkbox during heavy batch loads.

9 Current Limitations & Diagnostic Playbook

While the v3.0.0 pipeline is highly accurate, edge cases in physical environments still cause matching discrepancies. We classify these into two distinct error archetypes:

9.1 Error Type A: The “Ghost” (Detection Failure)

Symptom: A student walks through the door, but the AI never draws a bounding box around them, and they are entirely missing from the CSV report.

- **Root Causes:**
 - *Occlusion:* Walking directly behind a taller person blocks the camera’s line of sight.

- *Trajectory Speed*: Moving so rapidly across the frame that the tracker fails to capture the required `MIN_SAMPLES = 4` frames needed to establish a stable track.
- *Head Pose*: Turning the head 90 degrees away from the camera causes YOLOv8-face to drop the detection.
- **Proposed Solutions**: Physically repositioning cameras to avoid natural choke-points, or dynamically lowering the YOLO confidence threshold in well-lit areas to catch partial facial profiles.

9.2 Error Type B: The “Visitor Mismatch” (Recognition Failure)

Symptom: The student is perfectly tracked and given a `G_ID`, but when their embedding is compared to the S3 master database, the system labels them as a `visitor` or `returning_employee`.

- **Root Causes**: The cosine similarity score fell below our strict threshold (e.g., `T_GOOD_MATCH = 0.40`). This happens if the student grew facial hair, wore heavy glasses, or if the S3 master photo is outdated, poorly lit, or blurry.
- **Proposed Solutions**:
 - *Threshold Tuning*: We can relax `T_SUSPECT` to 0.30, catching more students but risking False Positives (labeling an actual visitor as a student).
 - *The Visitor Gallery Audit*: Run a script to extract crops of all people marked as “visitors”. If human review identifies enrolled students in this gallery, trigger a UI prompt for that student to upload a fresh S3 master photo.

10 Milestones, Successes, & Key Learnings

10.1 Project Milestones

1. **Pipeline Maturation:** Successfully transitioned from a volatile ByteTrack pipeline to a highly stable, Re-ID enabled BoTSORT architecture.
2. **Zero-Touch Automation:** Deployed the idempotent folder-listening mechanism, allowing the software to ingest new .mp4 files seamlessly without reprocessing old ones.
3. **Cloud Analytics Sync:** Successfully deployed the BigQuery API integration, enabling automated flattening and uploading of complex, multi-site JSON attendance data.
4. **The Ultimate Audit Pipeline:** Engineered the Colab-based 6-second video clip generator, bridging Google Drive, BigQuery, and AWS S3 into a single verification workflow.

10.2 Failures & Strategic Pivots

- **Failure:** Relying strictly on NVR file metadata start-times for frame calculation. This completely failed due to variable frame rates and metadata lag, resulting in incorrect frame extraction.
- **Pivot:** Generating 6-second temporal buffers instead of single frames, transferring the burden of exact timing from the flawed metadata to the human auditor reviewing the clip.
- **Failure:** Executing massive I/O operations (saving hundreds of face crops) directly to a mounted Google Drive in Colab.
- **Pivot:** Utilizing Colab's ephemeral local storage as a high-speed cache, followed by a single ZIP and transfer operation.

10.3 Key Engineering Learnings

- **Data Sanitization is Paramount:** Assuming upstream data is clean causes pipeline collapse. BigQuery treating dates as text strings and AWS S3 treating folders as 0-byte files taught us to implement aggressive type-checking and filtering (e.g., `PARSE_DATE` and `.endswith('.jpg')`).
- **Visual Verification Over Blind Metrics:** In computer vision, relying purely on precision/recall metrics from logs is dangerous. Building the visual side-by-side Colab dashboard exposed the NVR time-drift issue that logs would have never caught.

11 Future Scope & Roadmap

11.1 1. Native OCR Timestamp Synchronization

To permanently solve the NVR time-drift issue, the pipeline should integrate **EasyOCR**. Instead of calculating frames based on the `.mp4` file creation time, the system can periodically read the timestamp burned directly into the video pixels (e.g., the top-right corner of the CCTV feed). This grounds the tracking logic in the camera’s absolute physical reality.

11.2 2. Automated Re-Enrollment (The “Frequent Visitor” Pipeline)

Currently, tracking failures require manual auditing of the visitor logs. We propose an automated module: if a specific unknown face (visitor) is tracked across 3 consecutive days with high embedding consistency, the UI should flag them in a “Potential Student” dashboard. Center administrators can click a button to assign this face to a student ID, automatically updating the S3 master gallery with their current appearance.

11.3 3. Edge-to-Cloud Inference Migration

As TataStrive scales to hundreds of classrooms, relying on edge PCs (which require PyInstaller updates and hardware maintenance) becomes restrictive. The future architecture should involve lightweight edge nodes that merely chunk and securely stream video to a centralized Cloud GPU cluster (e.g., GCP Vertex AI or AWS SageMaker) where the heavy YOLO/BoTSORT models operate elastically.

A Access Management & Quick Start

Before running the pipelines or audit scripts mentioned in this document, a new developer, intern, or IT administrator must secure the following credentials and permissions.

1. **Code Repository:** Ensure you have Git access to the `TataStriveFinal` repository to pull the latest `app/`, `installer/`, and `scripts/` directories.
2. **AWS S3 Credentials:** You will need an `AWS_ACCESS_KEY` and `AWS_SECRET_KEY` with read-access to the `tata-strive-student-roster` bucket. To configure this locally, run `aws configure` in your terminal. For Google Colab, add these keys to your Colab Secrets environment.
3. **Google Drive Access:** Ensure your corporate Google account has shared access to the `/output` folders containing the center videos and JSON reports.
4. **Google Cloud (BigQuery):** Request IAM permissions for the `tatastrive_project` to view and query the `attendance_reports` and `engagement_reports` tables.

B Operator Runbook: New Center Deployment

When onboarding a new TataStrive physical training center, IT administrators should follow this standardized deployment runbook to ensure the edge PC is correctly configured to run the AI pipeline.

B.1 Phase 1: Environment & Hardware Verification

- **OS Verification:** Ensure the edge PC is running Windows 10 or 11 (64-bit).
- **Visual C++ Redistributable:** Install the latest Microsoft Visual C++ Redistributable. This is a strict requirement; without it, ONNX Runtime and PyTorch DLLs will fail to load, and the analysis tabs will be disabled.
- **Camera Feed Mapping:** Ensure the center's NVR (Network Video Recorder) is mapped to a local network drive on the edge PC (e.g., Z:\Classroom Feeds\).

B.2 Phase 2: Installation & Configuration

1. **Run the Installer:** Execute the latest `TataStriveAnalytics.Setup_v3.0.0.exe` (generated via Inno Setup).
2. **First Launch Initialization:** Open the application. Upon first launch, the system will prompt for a **Center ID**. Enter the official site tag (e.g., `Mumbai.Center.01`). This tag is permanently bound to all BigQuery uploads.
3. **Model Weights Initialization:** Ensure the `Models/` directory is populated with the following files:
 - `yolov8m.pt` (Person Detection)
 - `yolov8n-face.pt` (Face Detection)
 - `osnet_x1_0_msmt17.pt` (BoTSORT Re-ID)

B.3 Phase 3: Pipeline Activation

1. Navigate to the **Attendance Only** tab (Ctrl+2).
2. **Video Input Folder:** Browse and select the mapped NVR drive.
3. **Database File:** For Day 1, leave this blank. The system will automatically run in `BUILD_DB` mode to generate the initial `.pk1` master gallery. For subsequent days, point this to the generated gallery file (`EVAL_DAY` mode).
4. **Output Directory:** Select a local high-speed SSD path for JSON report generation.
5. Click **Start Monitoring**. Verify in the log panel that the "Folder Listener" has started and is detecting `.mp4` files.

C Core Configuration Schema (config.json)

The behavior of the TataStrive Analytics application is entirely governed by the `config.json` file generated in the operator's user profile (`~/tatastrive/config.json`). Below is a reference for the critical schema keys used to tune the AI's performance.

```
1 {
2   "center_id": "Mumbai_Center_01",
3   "bigquery": {
4     "auto_sync": true,
5     "sync_hour_utc": 18,
6     "last_sync_date": "2026-06-18"
7   },
8   "classroom": {
9     "default_probe_duration": 120,
10    "default_probe_interval": 3600,
11    "default_frame_skip": 5,
12    "stitching_max_time_gap": 300,
13    "stitching_max_pixel_dist": 150,
14    "delete_source_after_processing": false
15  },
16  "crossday": {
17    "t_good_match": 0.40,
18    "t_suspect_match": 0.30,
19    "min_tracking_samples": 4,
20    "generate_annotated_video": true
21  },
22  "inference": {
23    "device": "cpu",
24    "yolo_input_size": 640,
25    "face_input_size": 640
26  },
27  "crossday_completed_videos": [
28    "Z:\\\\Classroom_Feeds\\\\NVR_ch4_main_20260406090515.mp4"
29  ]
30 }
```

Listing 3: Sample config.json Schema

Configuration Tuning Notes for IT:

- **Hardware Bottlenecks:** If the edge PC experiences CUDA Out-Of-Memory errors, change `inference.device` to "cpu" and increase `classroom.default_frame_skip` to 10 or higher.
- **Force Reprocessing:** The `crossday_completed_videos` array is what makes the application idempotent. If a video must be forcefully re-processed (e.g., due to a crash halfway through), IT can manually delete the specific file path string from this array, save the file, and restart the application.